

Technical Paper

DRIFT

Distributed Routing for Inference and Feature Tasks

A Serverless, Peer-to-Peer Framework for Capability-Based LLM Inference Routing across
Heterogeneous Consumer Hardware

Kaan Yedikardaşlar

Version 0.1 · 2025

. Abstract

The widespread availability of consumer-grade GPU hardware has created an untapped reservoir of local compute capacity. Yet existing frameworks for Large Language Model (LLM) inference either require a centralized server, demand manual cluster configuration, or approach distributed execution by partitioning a single model across devices — an approach that couples hardware tightly and limits flexibility.

This paper introduces DRIFT (Distributed Routing for Inference and Feature Tasks), a serverless, peer-to-peer framework that reconceptualizes distributed LLM inference through capability-based task routing. Rather than splitting a model, DRIFT treats each physical machine (DNODE) as an autonomous worker that advertises its capabilities (DCAP) to a self-organizing network (the Hive). When a job arrives, every eligible DNODE submits a scored bid; the node with the highest available capacity wins and executes the task locally. New nodes join by simply launching the process — zero configuration required.

We describe the architecture, the SSHR scoring mechanism, the bidding and consensus protocol, and discuss how DRIFT differs from existing work. We also identify open challenges and directions for future development.

1. Introduction

Modern LLM inference is dominated by two paradigms: cloud-hosted APIs and single-machine local execution. Both carry significant limitations. Cloud APIs introduce latency, ongoing cost, and data privacy concerns. Single-machine inference is constrained by the VRAM ceiling of one GPU, making large models inaccessible to most individuals.

Meanwhile, many homes, small offices, and research labs contain multiple machines with discrete GPUs that sit largely idle. The aggregate VRAM across these machines often exceeds what any single node provides, yet no practical framework exists to harness this capacity in a lightweight, serverless manner.

DRIFT addresses this gap. Its central insight is that distributed inference does not require model partitioning. If each machine runs a complete, independent model or specialized service, then a coordination layer that routes tasks to the best available node is sufficient — and architecturally simpler. This approach draws inspiration from how skilled workers in an organization self-assign to tasks they are qualified for, weighted by their current availability.

The contributions of this paper are:

- A formal description of the DRIFT architecture and its core terminology (DNODE, DCAP, SDNODE, Hive).
 - The SSHR (Super Selective Hardware Router) scoring function and its rationale.
 - A bidding and consensus protocol for decentralized job assignment.
 - A comparison with existing distributed inference systems.
 - A reference implementation in Python using UDP broadcast.
-

2. Background and Related Work

2.1 Existing Distributed Inference Systems

Several frameworks have explored distributed LLM inference, but each takes a fundamentally different approach from DRIFT.

Petals (Borzunov et al., 2022) enables collaborative inference by distributing transformer layers across volunteer nodes connected over the internet. It requires a tracker server and is optimized for a single large model split across many participants. Exo (2024) enables peer-to-peer inference across heterogeneous consumer devices using UDP broadcast for discovery and pipeline parallelism for execution. While architecturally closer to DRIFT, Exo focuses on partitioning a single model rather than routing independent tasks to specialized nodes. EdgeShard (Zhang et al., 2024) partitions LLMs across heterogeneous edge and cloud devices using dynamic programming for optimal placement, targeting IoT environments rather than general-purpose local clusters.

In the routing literature, systems such as Hybrid LLM, Eagle, and GraphRouter focus on routing queries between models of different sizes or quality levels in cloud environments. These approaches are centralized and do not address hardware-level heterogeneity or zero-configuration deployment.

2.2 Key Differentiator

DRIFT occupies a distinct position: it is the only framework, to our knowledge, that combines (a) zero-configuration P2P discovery, (b) capability-based task routing rather than model partitioning, (c) hardware-aware scored bidding, and (d) a pluggable specialization model via DCAP tags — all without a central server.

3. DRIFT Architecture

3.1 Core Terminology

Term	Full Form	Definition
DRIFT	<i>Distributed Routing for Inference and Feature Tasks</i>	The framework and protocol as a whole.

DNODE	<i>Distributed Node</i>	Any physical machine participating in the Hive. Runs one or more DCaps.
SDNODE	<i>Super Distributed Node</i>	A DNODE with wildcard capability (*). Eligible for all job types.
DCAP	<i>Distributed Capability</i>	A capability label advertised by a DNODE (e.g. llm, tts, vision, embeddings).
Hive	—	The self-organizing network of all active DNODEs.
SSHR	<i>Super Selective Hardware Router</i>	The scoring and routing engine that selects the winning DNODE per job.

3.2 System Overview

DRIFT operates as a fully decentralized, flat network. There is no master node, no registry server, and no shared state store. All coordination happens through broadcast messages over a local network (or VPN overlay for wider topologies).

A DRIFT Hive consists of:

- One or more DNODEs, each running the `drift_node` process.
- One or more Clients, each running the `drift_client` process.
- A shared UDP broadcast channel on a configurable port (default: 50000).

3.3 Node Lifecycle

Discovery Phase

On startup, each DNODE immediately begins broadcasting a discovery message at a fixed interval (default: 5 seconds). The message contains the node's unique identifier, its list of DCaps, and its total GPU VRAM.

```
{ "type": "discovery", "node_id": "a3f9c1b2", "capabilities": ["llm",
"tts"], "vram": 12.0 }
```

All other DNODEs and Clients receiving this message update their local Hive map. A node is considered active as long as a discovery message has been received within a configurable TTL window.

Job Arrival

When a Client broadcasts a job, it specifies the required task type and an opaque payload. Every DNODE that holds the matching DCAP (or is an SDNODE) becomes eligible.

```
{ "type": "job", "job_id": "b7d2e0a1", "task_type": "llm", "payload": {  
... }, "client_id": "f1c3a9d4" }
```

Bidding

Each eligible DNODE computes its SSHR score (see Section 3.4) and broadcasts a bid message. Bids are collected during a configurable bidding window (default: 2 seconds).

```
{ "type": "bid", "job_id": "b7d2e0a1", "node_id": "a3f9c1b2", "score":  
142.7, "vram": 12.0 }
```

Consensus and Execution

After the bidding window closes, each DNODE independently evaluates the full set of received bids. The node with the highest score wins. Because all nodes apply the same deterministic rule (max score wins), consensus is reached without a coordinator. The winning node executes the job locally and broadcasts a `start_job` notification so the Client and other DNODEs can update their state.

If two nodes receive identical scores (a tie), the node with the lexicographically smaller `node_id` wins. This rule is deterministic and requires no additional message exchange.

4. The SSHR Scoring Function

4.1 Design Goals

The scoring function must be fast (no network calls), hardware-aware, and produce a total ordering over all bidding nodes. It should reward nodes with more available compute and penalize nodes currently under load.

4.2 Formula

The base SSHR score for a given node at bid time is:

$$\text{score} = (\text{VRAM_total} \times 10) + (\text{RAM_available} \times 2) - \text{CPU_load_percent}$$

If the DNODE holds an exact DCAP match for the incoming task type, the score is multiplied by a specialization bonus (default: 1.5). This ensures that a node specifically configured for a task type will win over a generic SDNODE with equivalent hardware, all else being equal.

```
if task_type ∈ DCAP → score × 1.5
```

The score is clamped to a minimum of 0.1 to prevent zero bids from CPU-only nodes from being treated as non-participants.

4.3 Rationale

Component	Rationale
VRAM × 10	VRAM is the dominant constraint for LLM inference. It is weighted heavily to ensure the node with the most GPU memory wins by default.
RAM_available × 2	Available (not total) RAM reflects actual headroom. A node with 64GB total but 2GB free is a poor candidate.
– CPU_load	High CPU load indicates the machine is busy. The penalty discourages routing to overloaded nodes even if their VRAM is large.
× 1.5 (specialization)	A node configured specifically for a task type has likely loaded the relevant model or service. Rewarding specialization avoids unnecessary model-switching on generic nodes.

5. Protocol and Fault Tolerance

5.1 Message Types

Message Type	Description
discovery	Periodic heartbeat broadcast by each DNODE. Contains node_id, capabilities, vram.
job	Broadcast by Client to announce a new task. Contains job_id, task_type, payload.

bid	Broadcast by eligible DNODEs. Contains job_id, node_id, score, vram.
start_job	Broadcast by the winning DNODE after consensus. Notifies Client and Hive of assignment.

5.2 Fault Scenarios

DNODE failure before bid

If a DNODE crashes before bidding, it simply does not appear in the bid pool. The remaining eligible nodes proceed to consensus normally. No recovery action is needed.

DNODE failure after winning

If the winning node crashes after broadcasting start_job but before completing the task, the Client receives no result. In the current implementation, the Client is responsible for timeout detection and job resubmission. A future version will implement an acknowledgment-and-watchdog mechanism at the DNODE level.

Network partition

In a split-brain scenario where the network partitions mid-bidding, two nodes may independently conclude they have won. This is an acknowledged limitation of the current protocol. A future iteration will introduce a lightweight commit phase: the winning node broadcasts a claim message and waits for a quorum of acknowledgments before beginning execution.

Stale Hive entries

Each Hive entry is timestamped on receipt. A DNODE that has not broadcast a discovery message within a configurable TTL is considered offline and removed from the local Hive map. This prevents routing decisions from considering nodes that are no longer reachable.

6. Comparison with Existing Systems

Property	Petals	Exo	EdgeShard	DRIFT
----------	--------	-----	-----------	-------

Approach	Layer sharding	Model sharding	Dynamic placement	Capability routing
Central server	Required	Not required	Not required	Not required
Zero-config join	No	Partial	No	Yes
Specialization model	None	None	None	DCAP tags
Target hardware	Internet volunteers	Consumer local	IoT / edge	Any local hardware
Production readiness	Research	Experimental	Research	In development

7. Reference Implementation

7.1 Overview

The reference implementation is written in Python 3 and uses only the standard library plus psutil for hardware metrics and GPUtil for GPU detection. No broker, database, or orchestration runtime is required.

Two entry points are provided:

- `drift_node.py` — starts a DNODE. The `--caps` flag sets the DCAP list. Omitting it starts an SDNODE.
- `drift_client.py` — starts a Client UI that broadcasts jobs and displays assignment results.

7.2 Starting a Hive

To start a three-node Hive with two specialized nodes and one SDNODE, run the following on three separate machines (or terminal sessions on one machine for testing):

```
python drift_node.py --caps llm
python drift_node.py --caps tts
python drift_node.py
```

Then start the Client:

```
python drift_client.py
```

All nodes discover each other automatically within one announcement cycle (5 seconds by default). No IP addresses, hostnames, or configuration files need to be specified.

7.3 Configuration Constants

Constant	Default / Description
BROADCAST_PORT	50000 — UDP port for all Hive communication.
BIDDING_WINDOW	2.0s — Time to collect bids before consensus.
JOB_TTL	30.0s — Time before a completed job is removed from memory.
ANNOUNCE_INTERVAL	5.0s — Frequency of DNODE heartbeat broadcasts.
CAPABILITY_MATCH_BONUS	1.5× — Score multiplier for exact DCAP match.

8. Current Limitations and Future Work

8.1 Known Limitations

- Split-brain consensus: Two nodes may simultaneously conclude they have won if bids are lost due to network conditions. A commit-and-quorum phase is planned.
- No result delivery protocol: The current implementation simulates job execution. A real payload delivery channel (e.g. direct TCP or gRPC) between the winning DNODE and the Client is not yet implemented.
- Fixed bidding window: The 2-second window is suitable for LAN environments but may be too short or too long for other topologies. Adaptive windowing based on observed peer latency is a planned improvement.
- No authentication or encryption: All messages are plaintext UDP. For production use, a shared-key HMAC or mTLS overlay is required.
- GPU detection dependency: GPUUtil is an optional dependency. Without it, VRAM is simulated. Native detection via nvml or a cross-platform abstraction layer is planned.

8.2 Future Work

- Commit-and-quorum consensus phase to eliminate split-brain.

- Streaming result delivery over TCP/WebSocket from winning DNODE to Client.
 - Web-based Hive dashboard showing live DNODE status, DCAP map, and job history.
 - WAN support via optional VPN overlay (e.g. Tailscale, ZeroTier) with relay-aware routing.
 - Dynamic DCAP loading: hot-plug a new model onto a running DNODE without restart.
 - Formal evaluation: latency benchmarks, throughput under varying Hive sizes, and fault injection tests.
-

9. Conclusion

DRIFT presents a new approach to distributed LLM inference: instead of partitioning a model across machines, it routes complete tasks to the most capable available machine. The framework is designed around three principles — zero-configuration join, capability-based specialization, and hardware-aware autonomous bidding — that together make local multi-machine AI infrastructure practical for individuals, small teams, and researchers.

The reference implementation demonstrates that the core protocol can be realized in under 300 lines of Python, with no external dependencies beyond hardware introspection libraries. We believe DRIFT fills a genuine gap in the existing landscape and invite contributions, experimentation, and critique from the community.

. References

- [1] Borzunov, A. et al. (2022). Petals: Collaborative Inference and Fine-tuning of Large Models. arXiv:2209.01188.
- [2] Exo (2024). Run your own AI cluster at home with everyday devices. github.com/exo-explore/exo.

[3] Zhang, M. et al. (2024). EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. IEEE Internet of Things Journal.

[4] Ong, I. et al. (2024). RouterBench: A Benchmark for LLM Routing. arXiv:2403.12031.

[5] Distserve (2024). Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. OSDI 2024.

[6] psutil. Cross-platform library for retrieving process and system information. github.com/giampaolo/psutil.

[7] GPUtil. GPU monitoring library for Python. github.com/anderskm/gputil.